

讲义

September 2, 2026

1 Python 数据数学分析核心讲义

1.1 引言: 从工程代码回归数据本质

1.1.1 为什么是 Python?

在传统的软件后端开发中, 关注点通常是 **状态管理、网络 I/O、并发安全与面向对象的解耦抽象**. 然而, 当进入数据分析与科学计算领域时, 思维模型需要发生根本性转变:

[传统后端工程思维]	[现代数据分析/矩阵思维]
对象封装(OOP)	-> 数据与操作分离(Data-Oriented Design)
逐个遍历(for item in list)	-> 批量向量化计算(SIMD / Tensor Processing)
复杂控制流(if-else / 状态机)	-> 布尔掩码与索引选择(Mask & Fancy Indexing)
显式多线程/锁竞争	-> 底层 C/Fortran/BLAS 内存连续并行计算

Python 之所以能够统治现代数据科学与 AI 领域, 并非因为其解释器运行效率高 (相反, CPython 的解释器开销与 GIL 限制众所周知), 而是因为它的 **“胶水特性”与 C-API 扩展能力**:

1. 上层提供极具表现力的动态语法和交互式 REPL 环境;
2. 下层无缝对接 OpenBLAS、MKL、CUDA 等底层硬件加速库;
3. 形成了从数据摄取 (Pandas/Arrow)、处理 (NumPy/SciPy)、可视化 (Matplotlib/Seaborn) 到模型训练 (PyTorch/Scikit-learn) 的完整闭环.

为什么 Python 明明什么都能算, 我们还需要 NumPy 和 Pandas?

NumPy 和 Pandas 真正带来的价值, 是“多了一堆 API”, 还是“改变了数据和计算的抽象方式”?

1.1.2 我们真正要学的不是 API

假设现在拿到 200 万条出租车订单.

我们需要分析:

1. 哪些订单不可信?
2. 什么时候最忙?
3. 什么时候流水最高?
4. 哪些区域最热门?
5. 哪些订单看起来异常?
6. 为什么同一个计算, 有人的代码跑几十秒, 有人的代码只跑几秒?
7. 怎么把这些分析变成公司里以后还能复用的工具?

当然, 可以全部使用 Python:

```
for row in rows:
    ...
```

但问题并不在于 Python 能不能做.

真正的问题是:

Python 原生的数据结构和逐对象执行模型, 并不是专门针对百万级同构数值计算和二维表格分析设计的.

但是 Numpy 和 Pandas 是 Python 中专门为数据分析设计的

- 1. NumPy 的核心是同构 N 维数组 `ndarray`; 官方文档将其定义为 NumPy 的核心多维数组结构, 并围绕数组提供高效的数学、逻辑、选择、排序、线性代数等运算.
- 2. Pandas 则在数组之上增加了 **标签、索引、缺失值、异构列、自动对齐、分组、连接和时间序列**等更贴近表格数据分析的抽象; `Index` 本身就是 Pandas 用于索引和对齐的轴标签对象.

从哪里来	核心技术	关键机制	最终走向
Python 原生对象	<code>list / dict / Object</code>	灵活, 但对象开销大、逐元素处理	↓
NumPy	<code>ndarray</code>	<code>dtype、shape、strides</code>	高效数值计算
NumPy 计算模型	<code>UFunc / Vectorization / Broadcasting</code>	批量计算、底层循环、形状扩展	性能提升
Pandas	<code>Series / DataFrame</code>	<code>Index、Alignment</code>	结构化数据分析
Pandas 数据操作	<code>groupby / merge / resample / rolling</code>	聚合、关联、时间序列、窗口计算	复杂数据处理
业务分析	数据清洗 → 分析 → 建模	将工具应用于真实数据	业务结论
性能工程	内存、向量化、拷贝、计算瓶颈	针对大规模数据优化	更快、更省内存
领域工具 / Pipeline	专用工具、Accessor、Pipeline	封装通用能力	工程化应用

一句话概括: > NumPy 解决 “如何高效计算数组”; Pandas 解决 “如何带着业务语义高效组织、查询、组合和分析表格数据”.

1.1.3 出租车数据集简介

NYC TLC Yellow Taxi Trip Records 是纽约市出租车与豪华轿车委员会 (NYC Taxi & Limousine Commission, TLC) 公开发布的纽约市黄色出租车行程记录数据.

类别	字段	含义	典型值/单位
记录信息	<code>VendorID</code>	提供出租车计价/数据记录服务的技术供应商编号	1, 2, 6, 7
时间	<code>tpep_pickup_datetime</code>	计价器开始计费的时间, 即上车时间	2023-01-01 00:15:00
	<code>tpep_dropoff_datetime</code>	计价器停止计费的时间, 即下车时间	2023-01-01 00:25:00

类别	字段	含义	典型值/单位
乘客/行程	passenger_count	乘客人数	1, 2, 3...
费率	trip_distance	计价器记录的行程距离	英里 mile
数据传输	RatecodeID	本次行程最终使用的费率类型	1, 2, 3...
	store_and_fwd_flag	行程数据是否曾暂存在车载设备中后再上传	Y / N
空间位置	PULocationID	上车所在的 Taxi Zone	Zone ID
	DOLocationID	下车所在的 Taxi Zone	Zone ID
支付	payment_type	支付方式	信用卡、现金等
费用	fare_amount	计价器计算出的基础车费	美元
	extra	额外费用/附加费	美元
	mta_tax	MTA 税费	美元
	tip_amount	小费金额	美元
	tolls_amount	行程产生的过路费	美元
	improvement_surcharge	Taxi Improvement Surcharge	美元
	total_amount	向乘客收取的总金额	美元
	congestion_surcharge	拥堵附加费	美元
	airport_fee	JFK / LaGuardia 机场相关费用	美元
	cbd_congestion_fee	CBD 拥堵费, 2025 年起出现	美元

数据的格式

特性维度	CSV	Parquet
存储结构	行式存储 (Row-oriented, 纯文本)	列式存储 (Columnar, 二进制)
文件体积	较大 (无内置压缩, 字符冗余高)	极小 (内置 Snappy/ZSTD 等高效压缩与字典编码)
读取性能	较慢 (需逐行扫描、切分字符串并推断类型)	极快 (支持列裁剪与分区下推)
Schema & 类型	弱类型 (无元数据, 需解析器自行推断)	强类型 (内嵌 Schema、数据类型及统计信息)
可读性	极高 (文本编辑器、Excel 直接打开修改)	无法直接阅读 (需专用解析引擎)
I/O 优化	必须全表/全行扫描	仅读取查询所需的列和数据块 (Row Groups)

1.1.4 贯穿本节的七个问题

问题	业务问题	逐步引出的技术
Q1	这 200 万条数据可靠吗?	dtype、缺失值、mask、loc、query
Q2	一天中什么时候最忙?	datetime、groupby、agg

问题	业务问题	逐步引出的技术
Q3	什么时间段的流水和单位运营时间收入最高?	向量化、assign、groupby、派生指标
Q4	哪些上下车区域最热门?	value_counts、merge、连接校验
Q5	哪些订单和时间窗口可能异常?	where、select、quantile、transform、resample、rolling
Q6	为什么同一个分析可以相差一个数量级甚至更多?	Python loop、apply、NumPy、Numba、eval/query、内存优化
Q7	如何把分析脚本变成公司可以重复使用的工具?	pipe、纯函数、Accessor、领域 API

1.1.5 NumPy / Pandas 为什么存在

问题 假设我们有 200 万个金额数据:

```
prices = [...]
```

需要:

全部上涨 5%
再增加 3 元服务费
过滤异常价格
求平均值

直接使用 Python List 实现

```
# 直接使用 List 存储与计算
result = []
```

```
for price in prices:
    result.append(price * 1.05 + 3)
```

问题是我们把 **200 万次循环控制** 交给了 Python 层. 具体来说, Python 原生 List 在这种大批量数值计算场景下存在两大明显短板:

1. 存储方面

Python List 并不是一块连续的数值缓冲区, 而是一个 **对象引用数组**. 它存储的每一个元素都是一个独立的 Python 对象 (这里是 float), 每个对象都带有引用计数、类型指针等元信息. 以 200 万个金额为例:

```
prices = [42.5, 17.3, 58.9, ...] # 200 万个 float 对象
```

- List 本身只存 200 万个**指针**, 每个指针 8 字节 (64 位系统);
- 每一个 float 对象本身还要额外占用约 24 字节 (CPython 对象头);
- 于是 List 存储 200 万个浮点数的实际内存, 大约是相同规模 ndarray 的 **3~5 倍**.

List:

指针数组 (8B × 200 万) + 每个 float 对象 (24B × 200 万)
16 MB + 48 MB = 64 MB 左右

```
ndarray(float64):  
    连续缓冲区 (8B × 200 万)  
    16 MB
```

内存翻倍, 还意味着缓存命中率更低、CPU 需要跨越更多随机内存地址.

2. 计算方面

```
result = []  
for price in prices:  
    result.append(price * 1.05 + 3)
```

这段循环的执行, 每一步都发生在 **Python 解释器层**, 而不是底层机器码层:

- 每次 for 迭代都要经历字节码分发 (bytecode dispatch);
- 每次读取 price 都要做一次 **拆箱** (unboxing, 从 Python 对象还原成 C double);
- 每次 `price * 1.05 + 3` 结果要重新 **装箱** (boxing) 成新的 float 对象;
- 每次 `append` 还要调用方法、检查容量、可能触发列表扩容与内存重分配.

当循环次数上升到 200 万, 这些**解释器级开销被放大 200 万倍**, 成为无可忽视的性能瓶颈. 这也是为什么同样的计算, 纯 Python 循环往往比 NumPy 向量化写法慢一到两个数量级——**真正慢的不是”算”, 而是”每次都要让解释器来驱动”**.

使用 Numpy 和 Pandas 实现 NumPy:

```
result = prices * 1.05 + 3
```

Pandas:

```
result = (  
    df  
    .query("fare_amount > 0")  
    .groupby("pickup_hour")  
    .agg(avg_fare=("fare_amount", "mean"))  
)
```

1.1.6 Numpy 和 Pandas 的特点

NumPy 解决的痛点 (纯数值与张量计算): 在科学计算场景下, 原生 Python 的 list 存储机制会导致严重的内存碎片化 (频繁指针解引用) 和缓存未命中 (Cache Miss), 同时动态类型解释器导致循环极慢. NumPy 的诞生终结了这五大痛点: 解决内存空间碎片化、消除解释器动态类型开销、填补多维张量代数的空白、终结切片的高昂拷贝代价, 并打通底层 C/BLAS 和 SIMD 硬件加速生态.

Pandas 解决的痛点 (异构业务数据与关系代数): 真实世界的业务数据不是纯数学矩阵, 而是包含字符串、时间戳、布尔值的异构表格, 常存在缺失值和错位情况. Pandas 解决了: 1. 如何容纳异构数据类型 (字符串、数值混合). 2. 在连接表或对齐时如何依据业务“键 (Label)”而非纯“位置 (Index)”进行运算. 3. 如何像 SQL 一样使用 GroupBy、Join 等关系代数算子. 4. 如何优雅且不间断程序地处理 NaN 缺失数据.

核心设计哲学

NumPy 的灵魂核心:ndarray(N-dimensional array) ndarray(N-Dimensional Array) 的本质是“同构多维连续内存缓冲区的视图抽象 (N-dimensional Array / Tensor)”，矩阵 (Matrix) 是它在 2 维空间下的一个特例。它的设计哲学是：**在动态语言中实现静态语言级别的内存布局**。

- **元数据解耦与视图 (View):** ndarray 将数据的 Header(包含 shape、dtype、strides) 与底层的数据 Buffer(连续物理内存) 彻底分离。这使得切片、转置、Reshape 都只需要极小开销修改 Header 且不需要复制物理内存。
- **齐次强类型与 SIMD:** 数据块内类型必须一致，使得 CPU 能够跨步寻址 (索引 × 大小)，并直接投喂给硬件向量化指令。
- **面向数组编程:** 把底层长循环隐式推给 C 语言层面，上层仅做“张量形态”的声明式表达。

Pandas 的根本基石:Series 和 DataFrame 如果 NumPy 是“同质数学张量”，Pandas 则是“带标签的异构列式数据框”。* **列式存储架构:** DataFrame 本质是多个一维 Series 的字典集合。在底层，同类型的列会被组织为内存连续的块。列式存储使得“求整列均值”等分析操作在物理上读取连续内存，极其高效。* **标签驱动 (Label-Based Alignment):** 与 NumPy 严格的矩阵位置形状对齐不同，Series 和 DataFrame 的核心哲学在于“业务主键 (Index) 绑定”。两张表相加时，Pandas 会自动依据相同的 Index 标签进行匹配 (如果没有则视为 NaN)，哪怕它们数据行数或顺序完全不同。这规避了业务分析中最致命的错位问题。

1.2 4. 课程环境与初始化

1.2.1 Numpy 与 Pandas 的版本

本课程 Numpy 与 Pandas 需要安装与配套的版本如下：

Package	Version
matplotlib	3.9.4
numba	0.60.0
numpy	2.0.2
pandas	2.3.3
pyarrow	21.0.0
jupyterlab	4.5.10

```
[4]: from __future__ import annotations

import gc
import math
import os
import platform
import sys
import time
import timeit
from pathlib import Path
from urllib.request import urlretrieve

import matplotlib.pyplot as plt
import numba
import numpy as np
```

```

import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq
from numba import njit

print("Python :", sys.version.split()[0])
print("NumPy  :", np.__version__)
print("Pandas :", pd.__version__)
print("PyArrow:", pa.__version__)
print("Numba  :", numba.__version__)
print("OS      :", platform.platform())

pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

DATA_DIR = Path("data")
ARTIFACT_DIR = Path("artifacts")

DATA_DIR.mkdir(exist_ok=True)
ARTIFACT_DIR.mkdir(exist_ok=True)

TAXI_PARQUET_PATH = DATA_DIR / "yellow_tripdata_2025-12.parquet"

TARGET_ROWS = int(os.getenv("TAXI_ROWS", "2000000"))
BENCH_ROWS = int(os.getenv("TAXI_BENCH_ROWS", "100000"))
IO_ROWS = int(os.getenv("TAXI_IO_ROWS", str(TARGET_ROWS)))

RANDOM_SEED = 42

print(f"{TARGET_ROWS=:,}")
print(f"{BENCH_ROWS=:,}")
print(f"{IO_ROWS=:,}")

```

```

Python : 3.9.25
NumPy   : 2.0.2
Pandas  : 2.3.3
PyArrow: 21.0.0
Numba   : 0.60.0
OS      : macOS-15.7.7-arm64-arm-64bit
TARGET_ROWS=2,000,000
BENCH_ROWS=100,000
IO_ROWS=2,000,000

```

1.2.2 本课程统一 Benchmark 工具

```
[5]: def benchmark(
    cases: dict[str, callable],
    *,
    repeat: int = 5,
    number: int = 1,
    warmup: int = 1,
) -> pd.DataFrame:
    """
    对多个无参 callable 做简单 benchmark.

    注意:
    - 这不是严格的微架构性能实验工具;
    - 用于课堂比较“数量级”和相对趋势;
    - 正式性能报告应该固定 CPU、电源策略、数据、线程数和环境.
    """
    records = []

    for name, func in cases.items():
        for _ in range(warmup):
            func()

        gc.collect()

        times = np.asarray(
            timeit.repeat(
                stmt=func,
                repeat=repeat,
                number=number,
            ),
            dtype=np.float64,
        ) / number

        records.append(
            {
                "name": name,
                "median_ms": np.median(times) * 1000,
                "min_ms": times.min() * 1000,
                "max_ms": times.max() * 1000,
            }
        )

    return (
        pd.DataFrame(records)
        .sort_values("median_ms")
        .reset_index(drop=True)
```

```

)

def plot_benchmark(result: pd.DataFrame, title: str) -> None:
    ordered = result.sort_values("median_ms", ascending=True)

    plt.figure(figsize=(9, 4.5))
    plt.barh(ordered["name"], ordered["median_ms"])
    plt.xlabel("Median time (ms)")
    plt.ylabel("")
    plt.title(title)
    plt.tight_layout()
    plt.show()

def dataframe_memory_mb(df: pd.DataFrame) -> float:
    return df.memory_usage(index=True, deep=True).sum() / 1024**2

def file_size_mb(path: Path) -> float:
    return path.stat().st_size / 1024**2

```

1.3 5. NumPy: 真正理解 ndarray

通过这部分希望大家可以理解下面的问题:

ndarray 更快的 list

```

ndarray =
    data buffer
    + dtype
    + shape
    + strides

```

能够解释:

- dtype
- shape
- ndim
- size
- itemsize
- nbytes
- strides
- flags
- view
- copy
- reshape
- transpose
- axis

1.3.1 ndarray 是什么

ndarray 是 Numpy 中的一个基础类型, 它底层的简化定义如下:

```
typedef struct {
    PyObject_HEAD

    /* Number of dimensions */
    int nd;
    /* Shape of the array */
    npy_intp *dimensions;
    /* Strides of the array */
    npy_intp *strides;
    /* Pointer to the actual data */
    void *data;
    /* Data type descriptor */
    PyArray_Descr *descr;
    /* Base object, used for views */
    PyObject *base;
    /* Flags */
    int flags;
    /* Weak references */
    PyObject *weakreflist;
    /* ... other internal fields ... */
} PyArrayObject;
```

NumPy 官方 ndarray 文档把 `shape`、`strides`、`ndim`、`data`、`itemsize`、`nbytes`、`base`、`dtype` 等直接列为数组 (ndarray) 的核心属性; `strides` 表示沿每个维度前进一步需要跨过的字节数.

ndarray 核心组成	对应概念	含义
data buffer	数据缓冲区	真正存储数据的内存区域
dtype	数据类型	描述一个元素应该怎样解释, 例如 <code>int64</code> 、 <code>float32</code>
shape	形状	描述数组的 逻辑维度 , 例如 <code>(3, 4)</code> 表示 3 行 4 列
strides	步长	描述 逻辑坐标如何映射到实际内存地址

所以 ndarray 本质上可以理解为: 一块数据内存 (Data Buffer) + 一组描述这块内存如何被解释和访问的元数据 (`dtype`、`shape`、`strides`).

Python list

ptr → Python object

ptr → Python object

ptr → Python object

NumPy ndarray

metadata:

```
shape    = (4,)
dtype    = float64
strides  = (8,)
```

data buffer:

```
1.2      3.4      5.6      7.8
```

新建一个 ndarray 我们创建一个最基本的 ndarray 如下:

```
[7]: a = np.array([1, 2, 3, 4], dtype=np.int32)

print("array   :", a)
print("dtype   :", a.dtype)
print("shape   :", a.shape)
print("ndim    :", a.ndim)
print("size    :", a.size)
print("itemsize:", a.itemsize)
print("nbytes   :", a.nbytes)
print("strides  :", a.strides)

# Numpy 内置了很多初始化函数:
m = np.arange(12, dtype=np.int32).reshape(3, 4)

print("shape   :", m.shape)
print("dtype   :", m.dtype)
print("strides  :", m.strides)
print("flags:")
print(m.flags)
```

```
array   : [1 2 3 4]
dtype   : int32
shape   : (4,)
ndim    : 1
size    : 4
itemsize: 4
nbytes  : 16
strides : (4,)
shape   : (3, 4)
dtype   : int32
strides : (16, 4)
```

```

flags:
  C_CONTIGUOUS : True
  F_CONTIGUOUS : False
  OWNDATA : False
  WRITEABLE : True
  ALIGNED : True
  WRITEBACKIFCOPY : False

```

对于上面的 C-order int32 数组, 一个元素 4 bytes:

```
shape = (3, 4)
```

内存:

```
0 1 2 3 | 4 5 6 7 | 8 9 10 11
```

列方向移动一步: 4 bytes

行方向移动一步: $4 \times 4 = 16$ bytes

```
strides = (16, 4)
```

常见创建函数:

```

[8]: examples = {
    "array": np.array([1, 2, 3]),
    "arange": np.arange(0, 10, 2),
    "linspace": np.linspace(0, 1, 5),
    "zeros": np.zeros((2, 3)),
    "ones": np.ones((2, 3)),
    "empty": np.empty((2, 3)),
}

examples

```

```

[8]: {'array': array([1, 2, 3]),
      'arange': array([0, 2, 4, 6, 8]),
      'linspace': array([0. , 0.25, 0.5 , 0.75, 1. ]),
      'zeros': array([[0., 0., 0.],
                      [0., 0., 0.]]),
      'ones': array([[1., 1., 1.],
                     [1., 1., 1.]]),
      'empty': array([[0., 0., 0.],
                      [0., 0., 0.]])}

```

ndarray 的内存占用 我们用一个简单的例子介绍 ndarray 在存储数值上对比 List 的优势, 假设我们比较一百万个整数:

```
[20]: python_list = list(range(1_000_000))
numpy_array = np.arange(1_000_000, dtype=np.int64)

python_approx_bytes = (
    sys.getsizeof(python_list)
    + sum(sys.getsizeof(x) for x in python_list)
)

print(
    "Python list 近似占用:",
    f"{python_approx_bytes / 1024**2:,.2f} MB"
)

print(
    "NumPy 元素缓冲区:",
    f"{numpy_array.nbytes / 1024**2:,.2f} MB"
)
```

Python list 近似占用: 34.33 MB

NumPy 元素缓冲区: 7.63 MB

这里 Python 的统计是 CPython 对象大小近似, 而 `ndarray.nbytes` 统计的是元素缓冲区本身, 两者并不是完全相同口径; 这个实验的重点是理解:

Python list 存的是一组 Python 对象引用, 而 NumPy 可以把同类型数值紧密存储。

但是不要为了“NumPy 看起来高级”把所有东西都放进 NumPy.

例如:

```
np.array(
    [
        {"name": "Alice"},
        {"name": "Bob"},
    ],
    dtype=object,
)
```

此时底层仍然绕不开 Python object, 大量 NumPy 数值计算优势会消失.

ndarray 的维度 Axis 在 ndarray 中, **axis(轴)** 的本质是**多维数组维度的“索引编号”与“遍历方向”**. 它定义了多维数据在逻辑结构中的组织层级, 以及算子在内存中沿哪个方向进行遍历或折叠.

1. axis 的本质

- **维度的编号:** 一个 N 维数组拥有从 0 到 $N - 1$ 的轴 (支持负数索引, -1 代表最后一维).
- **与 shape 的直接映射:** `ndarray.shape` 为一个元组 $(d_0, d_1, \dots, d_{N-1})$, 其中 `axis=i` 对应的就是维度大小 d_i .
- **与嵌套列表层级的映射:** 最外层的中括号对应 `axis=0`, 每往里深入一层括号, `axis` 编号加 1.
- **底层内存步长 (Strides) 的具象化:** 在默认的 C-order 连续内存中, `axis=0` 跨度最大 (跳过一个切片/行), `axis=-1` 跨度最小 (内存地址连续相邻).

2. 指定 axis 到底意味着什么？

理解 axis 计算最关键的一句话：“沿着指定的 axis 方向移动并折叠，消除该维度”。

3. axis 的核心作用

- **统计与聚合 (Reduction)**: 控制 sum、mean、max、std 等函数的计算方向, 决定保留哪些维度、压缩哪些维度.
- **拼接与扩展 (Concatenation & Stacking)**:
 - np.concatenate([a, b], axis=0): 沿垂直方向拼接 (增加行数).
 - np.concatenate([a, b], axis=1): 沿水平方向拼接 (增加列数).
- **排序与累加 (Sorting & Scanning)**: 如 np.sort(arr, axis=1) 仅对每一行内部的元素排序, 行与行之间互不干扰.
- **维度转换与重排 (Transposition)**: 在图像处理 (如深度学习中 (H, W, C) 转 (C, H, W)) 中, 通过 np.transpose(arr, axes=(2, 0, 1)) 重新排列各轴顺序.

```
[10]: import numpy as np

# =====
# 1. 2D 数组基础聚合 (求和、均值、最值)
# =====
# 创建一个 3 行 4 列 的二维矩阵
# shape: (3, 4) -> axis 0 长度为 3, axis 1 长度为 4
data_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])

# 【axis=None】: 不指定轴, 对所有元素进行全局计算, 结果压缩为标量
total_sum = np.sum(data_2d) # 结果: 78, shape: ()

# 【axis=0】: 沿着行索引变化的方向 (纵向/跨行) 向下压缩, 消除第 0 维
# 运算过程: [1+5+9, 2+6+10, 3+7+11, 4+8+12]
col_sum = np.sum(data_2d, axis=0) # 结果: [15, 18, 21, 24], shape: (4,)

# 【axis=1】: 沿着列索引变化的方向 (横向/跨列) 向右压缩, 消除第 1 维
# 运算过程: [1+2+3+4, 5+6+7+8, 9+10+11+12]
row_sum = np.sum(data_2d, axis=1) # 结果: [10, 26, 42], shape: (3,)

# 【最值与索引】: 找到每一行的最大值所在列索引
row_max_idx = np.argmax(data_2d, axis=1) # 结果: [3, 3, 3], shape: (3,)

# =====
# 2. 2D 数组拼接 (Concatenation)
# =====
a = np.array([[1, 2], [3, 4]]) # shape: (2, 2)
b = np.array([[5, 6], [7, 8]]) # shape: (2, 2)

# 【axis=0 拼接】: 沿着行方向延伸 (垂直向下追加新行), 列数必须相同
concat_axis0 = np.concatenate([a, b], axis=0)
# 结果: [[1, 2], [3, 4], [5, 6], [7, 8]], shape: (4, 2)
```

```

# 【axis=1 拼接】:沿着列方向延伸 (水平向右追加新列), 行数必须相同
concat_axis1 = np.concatenate([a, b], axis=1)
# 结果: [[1, 2, 5, 6], [3, 4, 7, 8]], shape: (2, 4)

# =====
# 3. 3D 数组的多轴聚合
# =====
# 创建三维数组:2 个切片 (深度), 每个切片 3 行 4 列
# shape: (2, 3, 4) -> axis 0= 深度 (切片), axis 1= 行, axis 2= 列
data_3d = np.arange(24).reshape((2, 3, 4))

# 【axis=0】:跨切片压缩 (把第 0 个切片与第 1 个切片对应位置相加), 消除深度维
sum_ax0 = data_3d.sum(axis=0) # shape: (3, 4)

# 【axis=1】:切片内部跨行压缩 (每个切片内各列求和), 消除行维
sum_ax1 = data_3d.sum(axis=1) # shape: (2, 4)

# 【axis=2 / axis=-1】:切片内部跨列压缩 (每个切片内各行求和), 消除列维
sum_ax2 = data_3d.sum(axis=2) # shape: (2, 3)

# 【元组多轴压缩】:同时消除 axis 1 和 axis 2, 仅保留切片维度
sum_ax12 = data_3d.sum(axis=(1, 2)) # shape: (2,)

# =====
# 4. 轴变换与重排 (Transpose)
# =====
# 图像常见格式转换:从 (高度, 宽度, 通道) 转为 (通道, 高度, 宽度)
# 原始维度: axis 0=H(100), axis 1=W(200), axis 2=C(3)
image_hwc = np.zeros((100, 200, 3))

# 【np.transpose】:显式重排轴的索引顺序 (2, 0, 1) 代表将旧的 axis 2 移到最前
image_chw = np.transpose(image_hwc, (2, 0, 1)) # shape: (3, 100, 200)

# 【np.swapaxes】:仅交换任意两个指定的轴 (例如交换行和列)
swapped = np.swapaxes(data_2d, 0, 1) # (3, 4) -> (4, 3), 等价于二维转置

```

View、Copy 与 Strides 在 ndarray 中, Strides(步长) 是连接高维逻辑坐标与底层一维内存的“寻址导航器”, 而 View(视图) 与 Copy(副本) 是由此衍生的两种内存管理策略. NumPy 之所以能在处理海量数据时保持极高的计算性能, 本质就在于将“数据实体”与“元数据 (Metadata)”彻底解耦.

为什么必须放在一起讲?

View 的物理实现完全依赖于 Strides 的重新映射; 掌握了 Strides 的线性寻址逻辑, 就能一眼看穿为什么基础切片可以实现零拷贝 ($O(1)$ 复杂度), 而花式索引却必须退化为内存深拷贝 ($O(N)$ 复杂度).

1. 本质

- **双层架构解耦**: ndarray 由两部分构成: 保存纯字节流的**连续数据缓冲区 (Data Buffer)*** 与保存描述信息的 *** 元数据头 (Metadata Header)**(包含 shape、dtype、strides、data pointer、base 等).
- **Strides(步长) 的本质**: 一个元组 $(s_0, s_1, \dots, s_{N-1})$, 表示在内存字节流中沿 `axis=i` 移动一个索引单位需要跨越的**物理字节数 (Bytes)**.
- **View(视图) 的本质**: 创建了一个全新的元数据头, 其 data pointer 仍指向**同一个底层内存缓冲区**. 创建时间复杂度为 $O(1)$, 对视图的数据修改会直接同步到原数组 (产生副作用).
- **Copy(副本) 的本质**: 在堆内存中开辟了一块全新的独立数据缓冲区, 并将数据完整复制过去. 时间复杂度为 $O(N)$, 与原数组在物理内存上完全隔离.

2. 原理

- **内存线性寻址公式**: 任意多维逻辑索引 $(i_0, i_1, \dots, i_{N-1})$ 对应的底层字节偏移量为:

$$\text{Byte Offset} = \sum_{k=0}^{N-1} (i_k \times s_k)$$

- **View vs Copy 的判定准则**:
 - **产生 View**: 只要操作后的新数据子集能够通过**单组等差步长 (Strides)** 与**基地址偏移** 完全表达 (如切片 `arr[:, :2]`、转置 `arr.T`、维度压缩 `arr.squeeze()`), NumPy 就仅修改元数据返回 View.
 - **强制 Copy**: 一旦目标元素的内存分布**无法用固定的线性步长表达** (如花式索引 `arr[[0, 2]]`、布尔掩码 `arr[arr > 5]`), 或显式调用 `.copy()`, NumPy 必须分配新内存并产生 Copy.

3. 核心作用

- **极速性能 (Zero-Copy)**: 切片、翻转、转置等操作耗时与数组大小完全无关 (均在纳秒级完成), 避免大规模内存申请与数据搬运.
- **内存安全控制**: 显式调用 `.copy()` 切断与原数组的指针绑定, 防止多线程计算或数据预处理流水线中的意外内存污染.
- **内存连续性 (Contiguity) 管理**: ndarray 的数据是否按照 C 语言/行优先 (row-major) 的方式连续存储, 确保调用底层 BLAS/LAPACK 或 C/C++ 扩展时的 SIMD 向量化加速效率.
- **高级数据重组 (Strided Tricks)**: 通过手动操控 Strides, 无需分配额外内存即可实现图像滑动窗口 (Sliding Window)、重叠切片或卷积核展开.

```
[11]: import numpy as np

# =====
# 1. 观察内存元数据与 Strides(步长机制)
# =====
# 创建 int64 类型的 2D 数组 (每个元素占 8 字节)
# shape: (3, 4) -> 3 行 4 列
arr = np.array([
    [10, 11, 12, 13],
    [20, 21, 22, 23],
    [30, 31, 32, 33]
```

```

], dtype=np.int64)

# 内存存储的格式 [10, 11, 12, 13, 20, 21, 22, 23, 30, 31, 32, 33]

# itemsize: 单个元素占用的字节数
print("单个元素字节大小 (itemsize):", arr.itemsize) # 8

# strides: 沿各个 axis 移动 1 个步长跨越的字节数
# axis 0 (跨行): 跨过 4 个 int64 元素 = 4 * 8 = 32 字节
# axis 1 (跨列): 跨过 1 个 int64 元素 = 1 * 8 = 8 字节
print("Strides 元组 (bytes):", arr.strides) # (32, 8)

# 内存寻址计算验证: 获取 arr[1, 2] 的内存偏移量
# Offset = (1 * 32) + (2 * 8) = 48 字节 (即第 48 字节处的数据: 22)

# =====
# 2. View(视图) 的产生、检测与副作用
# =====
# 【基础切片 (Basic Slicing) 返回 View】: 无需搬运数据, 仅生成新元数据
sub_view = arr[0:2, 1:3] # shape: (2, 2)

# 【检测是否为 View】: 通过 .base 属性判断
# 若 .base 指向原数组或不是 None, 说明是 View
print("sub_view 是否共享内存:", sub_view.base is arr) # True

# 【副作用 (Side Effect)】: 修改 View 会直接改变原数组
sub_view[0, 0] = 999
print("原数组 arr[0, 1] 受到联动修改:", arr[0, 1]) # 999

# 【转置 (Transpose) 也是 View】: 仅交换 strides 和 shape, 耗时 O(1)
transposed = arr.T # shape: (4, 3), strides: (8, 32)
print("转置数组的 strides:", transposed.strides) # (8, 32)
print("转置是否为 View:", transposed.base is arr) # True

# =====
# 3. Copy(副本) 的产生场景与内存隔离
# =====
# 【显式深拷贝】: 开辟全新内存空间
arr_copied = arr.copy()
print("显式 copy 的 base:", arr_copied.base) # None (拥有独立内存)
arr_copied[0, 0] = -1
print("原数组 arr[0, 0] 未受影响:", arr[0, 0]) # 10

# 【高级索引 (Fancy Indexing) 强制返回 Copy】:
# 选取的元素非等差分布, 无法用单一 strides 表达, 必须深拷贝

```

```

fancy_indexed = arr[[0, 2], [1, 3]] # shape: (2,)
print("花式索引是否为独立 Copy:", fancy_indexed.base is None) # True

# 【布尔掩码索引 (Boolean Masking) 强制返回 Copy】:
bool_indexed = arr[arr > 20] # shape: (5,)
print("布尔索引是否为独立 Copy:", bool_indexed.base is None) # True

# =====
# 4. 连续性 (Contiguity) 与 Reshape 的陷阱
# =====
# 连续性检查: C_CONTIGUOUS 代表行优先连续存储
print("原数组 C 连续性:", arr.flags.c_contiguous) # True
print("转置数组 C 连续性:", transposed.flags.c_contiguous) # False

# 【Reshape 在内存连续时返回 View】:
reshaped_view = arr.reshape(2, 6)
# array([[ 10, 999, 12, 13, 20, 21],
#        [ 22, 23, 30, 31, 32, 33]])
print("连续数组 reshape 是否为 View:", np.shares_memory(arr, reshaped_view)) # True
    ↪ True

# 【Reshape 在内存不连续时被迫触发隐式 Copy】:
# transposed 内存不连续, 无法用一组新 shape/strides 映射旧内存, 被迫复制
reshaped_copy = transposed.reshape(2, 6)
# array([[ 10, 20, 30, 999, 21, 31],
#        [ 12, 22, 32, 13, 23, 33]])
print("非连续数组 reshape 是否产生 Copy:", not np.shares_memory(transposed, reshaped_copy)) # True
    ↪ reshaped_copy)) # True

# =====
# 5. Strides 进阶黑魔法: 零拷贝滑动窗口
# =====
from numpy.lib.stride_tricks import as_strided

# 构造 1D 时序序列: [0, 1, 2, 3, 4, 5]
series = np.arange(6, dtype=np.int64)

# 需求: 提取窗口大小为 3、步长为 1 的重叠滑动窗口
# 目标输出 shape: (4, 3) -> 4 个窗口, 每个窗口 3 个元素
# 目标 strides:
#   axis 0 (下一个窗口): 沿原序列移动 1 个元素 = 8 字节
#   axis 1 (窗口内移动): 沿原序列移动 1 个元素 = 8 字节
sliding_windows = as_strided(
    series,
    shape=(4, 3),

```

```

        strides=(8, 8)
    )

    print("零内存分配的滑动窗口矩阵:\n", sliding_windows)
    # 结果:
    # [[0, 1, 2],
    #  [1, 2, 3],
    #  [2, 3, 4],
    #  [3, 4, 5]]
    print("滑动窗口是否为原序列的 View:", sliding_windows.base is series)  # True

```

单个元素字节大小 (itemsz): 8
 Strides 元组 (bytes): (32, 8)
 sub_view 是否共享内存: True
 原数组 arr[0, 1] 受到联动修改: 999
 转置数组的 strides: (8, 32)
 转置是否为 View: True
 显式 copy 的 base: None
 原数组 arr[0, 0] 未受影响: 10
 花式索引是否为独立 Copy: True
 布尔索引是否为独立 Copy: True
 原数组 C 连续性: True
 转置数组 C 连续性: False
 连续数组 reshape 是否为 View: True
 非连续数组 reshape 是否产生 Copy: True
 零内存分配的滑动窗口矩阵:
 [[0 1 2]
 [1 2 3]
 [2 3 4]
 [3 4 5]]
 滑动窗口是否为原序列的 View: False

1.3.2 ndarray 的用法

广播机制 (Broadcasting) 在 ndarray 的算术运算中, **Broadcasting(广播机制)** 是 NumPy 在不复制数据的前提下, 自动拉伸和对齐不同形状数组的“隐式虚拟扩展引擎”. 它是向量化计算 (Vectorization) 的核心支柱, 让具有不同形状的张量能够在底层 C 语言级别无缝进行逐元素 (Element-wise) 运算.

1. 本质

- **零内存消耗的虚拟扩展:** 广播并不是在物理内存中真实复制数据填充数组, 而是通过将对应拉伸维度的步长 (Strides) 设为 0. 步长为 0 意味着逻辑索引在递增, 但底层指针停留在同一物理内存地址上, 实现 $O(1)$ 内存开销的“逻辑克隆”.
- **双重规约机制:** 广播定义了严格的维度靠右对齐与长度兼容法则, 将多维数组间的算术运算、比较运算和赋值操作彻底标准化.

2. 原理 (对齐双法则)

广播在底层严格按照以下两个阶段执行判断与形状推导:

- **法则一：维度补齐（靠右对齐，左侧补 1）**
 - 如果两个数组的维度数 (ndim) 不同，NumPy 会在维度较少的数组 shape 左侧补 1，直到两者维度数量一致。
 - 例：shape=(3,) 与 shape=(4, 3) 对齐时，前者先被补齐为 (1, 3)。
- **法则二：长度兼容（等于自身或等于 1）**
 - 从最后一个维度 (Trailing Dimension, 即最右侧轴) 开始向前逐轴比对。对于每个轴，两者的长度必须满足以下条件之一，否则抛出 ValueError：
 1. 两个维度的长度相等；
 2. 其中一个维度的长度为 1 (该维度会被虚拟拉伸到与另一个维度相同)。
- **输出形状判定：**最终输出结果在每个轴上的长度，等于参与运算各数组在该轴长度的最大值： $\max(d_{1,i}, d_{2,i})$ 。

3. 核心作用

- **消除显式 Python 循环：**避免书写多重嵌套循环，将计算下沉至底层 C 连续遍历，极大激发 CPU 的 SIMD 向量化指令集性能。
- **内存极致优化：**彻底淘汰 np.tile 或 np.repeat 等物理深拷贝复制数据的低效做法。
- **高维特征与批处理运算：**在机器学习与图像处理中，轻松实现全局特征去中心化、样本按权重缩放、网格生成 (Meshgrid) 以及成对距离矩阵 (Pairwise Distance) 计算。

```
[12]: import numpy as np

# =====
# 1. 基础场景:标量广播与 1D/2D 混合运算
# =====
# 标量与 2D 数组运算
# arr shape: (2, 3), 标量 5 被视为 shape: () -> 广播为 (1, 1) -> (2, 3)
matrix = np.array([[10, 20, 30], [40, 50, 60]])
res_scalar = matrix + 5
# 结果: [[15, 25, 35], [45, 55, 65]], shape: (2, 3)

# 1D 行向量向 2D 矩阵广播
# row_vec shape: (3,)
# 广播流程: (3,) -> 靠右对齐补 1 为 (1, 3) -> 沿 axis 0 虚拟复制为 (2, 3)
row_vec = np.array([1, 2, 3])
res_row = matrix + row_vec
# 结果: [[11, 22, 33], [41, 52, 63]], shape: (2, 3)

# =====
# 2. 维度重塑:利用 np.newaxis 实现列向广播
# =====
# col_vec 原始 shape: (2,)
col_vec = np.array([100, 200])

# 【错误尝试】:matrix + col_vec 会报错!
# 原因: matrix(2, 3) 与 col_vec(2,) 靠右对齐后为 (1, 2), 最后一维 3 和 2 不兼容
```

```

# 【正确做法】:通过 np.newaxis (或 None) 显式插入新轴, 使其 shape 变为 (2, 1)
# 广播流程: (2, 1) 与 (2, 3) 对齐 -> 沿 axis 1 虚拟复制为 (2, 3)
col_vec_2d = col_vec[:, np.newaxis] # shape: (2, 1)
res_col = matrix + col_vec_2d
# 结果:
# [[110, 120, 130],
#  [240, 250, 260]], shape: (2, 3)

# =====
# 3. 广播机制的底层物理验证 (步长为 0 的秘密)
# =====
# np.broadcast_to 允许显式观察广播后的元数据视图
vec = np.array([10, 20, 30], dtype=np.int64) # shape: (3,), strides: (8,)

# 将 (3,) 广播为 (4, 3) 的二维矩阵视图
broadcasted_view = np.broadcast_to(vec, (4, 3))

# 【观察 Strides 步长】:
# axis 0 跨行步长为 0 bytes(说明跨行移动时, 物理内存指针完全不动!)
# axis 1 跨列步长为 8 bytes(正常读取连续内存)
print("广播视图的 shape:", broadcasted_view.shape) # (4, 3)
print("广播视图的 strides:", broadcasted_view.strides) # (0, 8)
print("广播视图是否共享底层内存:", np.shares_memory(broadcasted_view.base, vec))
↪ # True

# =====
# 4. 双向扩展:(M, 1) 与 (1, N) 生成外积网格
# =====
# a shape: (3, 1)
# b shape: (1, 4)
# 广播流程: 两者共同扩展为 (3, 4) 的矩阵
a = np.array([[1], [2], [3]])
b = np.array([[10, 20, 30, 40]])

# 自动生成 3x4 的相加组合矩阵 (无需 itertools 或双重 for 循环)
grid_sum = a + b
# shape: (3, 4)
# 结果:
# [[11, 21, 31, 41],
#  [12, 22, 32, 42],
#  [13, 23, 33, 43]]

# =====
# 5. 实战教学:零循环计算所有样本两两欧氏距离

```

```
# =====
# 假设有 3 个 2 维坐标点, shape: (3, 2)
points = np.array([[0, 0], [3, 0], [0, 4]])

# 构造 (3, 1, 2) 与 (1, 3, 2) 两个张量
p_expand_1 = points[:, np.newaxis, :] # shape: (3, 1, 2)
p_expand_2 = points[np.newaxis, :, :] # shape: (1, 3, 2)

# 利用广播计算两两坐标差, shape 自动变为 (3, 3, 2)
diff = p_expand_1 - p_expand_2

# 在特征轴 (axis=-1) 上计算欧氏距离 sqrt(dx^2 + dy^2)
dist_matrix = np.sqrt(np.sum(diff**2, axis=-1))

# 得到 3x3 的对称距离矩阵, 全过程完全向量化且无循环
print("样本成对距离矩阵:\n", dist_matrix)
# 结果:
# [[0. 3. 4.]
#  [3. 0. 5.]
#  [4. 5. 0.]]
```

广播视图的 shape: (4, 3)

广播视图的 strides: (0, 8)

广播视图是否共享底层内存: True

样本成对距离矩阵:

```
[[0. 3. 4.]
 [3. 0. 5.]
 [4. 5. 0.]]
```

Vectorization 与 UFunc(通用函数) Vectorization(向量化) 是 NumPy 高效计算的编程思想/使用方式, 而 UFunc(通用函数) 是 ndarray 实现高效逐元素计算的核心机制之一. 它们结合“连续内存 + 固定 dtype + C 层循环 + CPU 向量化/SIMD”等一整套机制共同构建了 Numpy 的高性能计算体系. Vectorization(向量化) 是将原本由 Python 解释器执行的逐元素标量循环, 下沉至底层 C 语言连续内存块与 CPU SIMD 硬件指令的“去循环执行模式”; 而 UFunc(Universal Function, 通用函数) 则是封装了这种高效 C 语言内核、并自带广播与高阶聚合方法的“可调对象引擎”。

1. 本质

- **消除解释器开销 (Bypassing Interpreter Overhead):** 摆脱 Python 在每次循环中反复进行的动态类型检查 (Dynamic Type Resolution)、对象封包解包 (Boxing/Unboxing) 以及引用计数维护, 将计算交由预编译的 C 循环指针递增执行.
- **UFunc 对象的物理本质:** 一个内建了多重类型分发表 (Type Dispatch Table) 与连续内存迭代器 (NpyIter) 的 C 结构体对象 (PyUFuncObject), 它同时管理着一元算子 (如 sin、exp) 与二元算子 (如 add、multiply).
- **硬件级并行 (SIMD 并发):** 向量化使连续内存数据能够直接加载进现代 CPU 的向量寄存器 (如 AVX-512、AVX2、ARM NEON), 实现单指令周期内对多个浮点数的并发吞吐 (SIMD, Single Instruction Multiple Data).

2. 原理

- **类型分发 (Type Resolution):** 传入数组时, UFunc 根据输入的 `dtype` 匹配最精确的底层 C 函数指针 (如 `int64` 走整数加法指令, `float64` 走双精度浮点流水线).
- **天然嵌入广播:** 任何多元 UFunc 都会自动调用广播协议, 无需额外处理维度即可自动扩展输入形状.
- **方法派生矩阵 (UFunc Methods):** 任何二元通用函数 (Binary UFunc, 如 `np.add`、`np.multiply`、`np.maximum` 等) 都自动派生出 5 个高阶方法, 直接将简单的逐元素算子转化为强大的矩阵与序列计算引擎:
 1. `.reduce()`: 沿指定轴折叠压扁 (累加/累乘/最值筛选);
 2. `.accumulate()`: 沿指定轴保存中间累积状态 (前缀和/前缀积);
 3. `.outer()`: 两输入做笛卡尔积全组合外积运算;
 4. `.at()`: 无缓冲的就地离散索引更新 (支持重复索引累加);
 5. `.reduceat()`: 按指定的索引切片边界分段折叠汇总.

3. 核心作用

- **极速性能吞吐:** 通常比原生 Python 的 `for` 循环与列表推导式带来 50 至 500 倍的性能提升.
- **高阶函数式表达:** 用 `.outer()` 或 `.reduce()` 替代复杂的多层循环与动态列表收集, 大幅精简工程代码.
- **解决原地累加数据冲突 (Race Hazard):** `ufunc.at` 解决了常规花式索引 (Fancy Indexing) 在遇到重复索引时因写入缓冲 (Buffering) 导致累加丢失的致命问题.
- **灵活的接口封装:** 借助 `np.vectorize` 与 `np.frompyfunc`, 可将任意纯 Python 业务函数快速封装为兼容广播特性的伪向量化对象.

```
[13]: import numpy as np

# =====
# 1. 基础 UFunc 与标量底层向量化
# =====
x = np.array([0.0, np.pi / 2, np.pi], dtype=np.float64)

# 一元 UFunc (Unary UFunc): 底层直接调用 C 标准库数学函数并发求值
sin_x = np.sin(x) # 结果: array([0., 1., 0.]), shape: (3,)

# 二元 UFunc (Binary UFunc): 自动触发广播机制并在 C 循环中逐元素相加
a = np.array([1, 2, 3])
b = np.array([[10], [20]]) # shape: (2, 1)
sum_ab = np.add(a, b)      # 等价于 a + b, 广播为 shape: (2, 3)
# 结果:
# [[11, 12, 13],
#  [21, 22, 23]]

# =====
# 2. UFunc 高阶核心方法族 (以 np.add / np.multiply 为例)
# =====
arr = np.array([1, 2, 3, 4, 5])
```

```

# 【.reduce() 沿轴折叠】:逐元素应用二元算子直到维度被消除
# 等价于  $1 + 2 + 3 + 4 + 5$ , 底层比手写 sum 效率更高
sum_val = np.add.reduce(arr) # 结果: 15
prod_val = np.multiply.reduce(arr) # 结果: 120 (阶乘)

# 【.accumulate() 沿轴扫描累积】:输出每一次计算的中间状态 (前缀序列)
cumsum_arr = np.add.accumulate(arr) # 结果: [1, 3, 6, 10, 15]
cumprod_arr = np.multiply.accumulate(arr) # 结果: [1, 2, 6, 24, 120]

# 【.outer() 笛卡尔外积】:对所有元素对  $(a_i, b_j)$  进行二元运算
vec_1 = np.array([1, 2, 3])
vec_2 = np.array([10, 20, 30, 40])
# 生成乘法外积矩阵 (shape: 3x4), 无需手动 reshape 和广播
outer_prod = np.multiply.outer(vec_1, vec_2)
# 结果:
# [[ 10, 20, 30, 40],
#  [ 20, 40, 60, 80],
#  [ 30, 60, 90, 120]]

# 【.reduceat() 分段区间聚合】:在给定的索引断点之间进行局部 reduce
# 区间划分:  $[0:2] \rightarrow 1+2=3$ ,  $[2:4] \rightarrow 3+4=7$ ,  $[4:] \rightarrow 5$ 
step_sum = np.add.reduceat(arr, [0, 2, 4]) # 结果: [3, 7, 5]

# =====
# 3. ufunc.at() 解决重复索引原地累加陷阱
# =====
target_1 = np.zeros(5, dtype=np.int64)
indices = np.array([1, 1, 2, 1]) # 包含重复的索引 1

# 【常规花式索引的陷阱】:使用写入缓冲区, 重复索引只会覆盖赋值, 无法累加
target_1[indices] += 1
print("普通切片累加的错误结果:", target_1)
# 结果: [0, 1, 1, 0, 0] (索引 1 只被增加了 1 次, 因为赋值发生了覆盖)

# 【ufunc.at() 无缓冲原地更新】:每次操作直接作用于内存, 保证累加正确
target_2 = np.zeros(5, dtype=np.int64)
np.add.at(target_2, indices, 1)
print("使用 np.add.at 的正确累加结果:", target_2)
# 结果: [0, 3, 1, 0, 0] (索引 1 被精准累加了 3 次)

# =====
# 4. 自定义函数的向量化包装 (np.vectorize 与 np.frompyfunc)
# =====
# 定义一个含有 Python 标量分支判断的非线性函数

```

```

def custom_relu_gate(x_val, threshold):
    """ 带阈值的定制激活函数 (无法直接接收原生 ndarray) """
    if x_val > threshold:
        return x_val - threshold
    else:
        return 0.0

# 【使用 np.vectorize 包装】:提供广播语法糖 (注意:本质仍是 Python 循环)
vec_gate = np.vectorize(custom_relu_gate, otypes=[np.float64])

inputs = np.array([1.0, 5.0, 3.0, 8.0])
thresh = 3.0
# 自动支持 ndarray 与标量广播输入
output = vec_gate(inputs, thresh) # 结果: [0.0, 2.0, 0.0, 5.0]

# 【使用 np.frompyfunc 构造通用函数】:生成标准的 UFunc 对象
# 参数依次为: (函数, 输入参数个数, 返回值个数)
pyfunc_gate = np.frompyfunc(custom_relu_gate, 2, 1)
# 产出的对象直接继承 UFunc 方法族
res_outer = pyfunc_gate.outer(inputs, np.array([2.0, 4.0])) # shape: (4, 2)

```

普通切片累加的错误结果: [0 1 1 0 0]

使用 np.add.at 的正确累加结果: [0 3 1 0 0]

1.4 Pandas:

我通过前面抛出的几个问题来解释 Pandas 的实际用法.

问题	业务问题	逐步引出的技术
Q1	这 200 万条数据可靠吗?	dtype、缺失值、mask、loc、query
Q2	一天中什么时候最忙?	datetime、groupby、agg
Q3	什么时间段的流水和单位运营时间收入最高?	向量化、assign、groupby、派生指标
Q4	哪些上下车区域最热门?	value_counts、merge、连接校验
Q5	哪些订单和时间窗口可能异常?	where、select、quantile、transform、resample、rolling
Q6	为什么同一个分析可以相差一个数量级甚至更多?	Python loop、apply、NumPy、Numba、eval/query、内存优化
Q7	如何把分析脚本变成公司可以重复使用的工具?	pipe、纯函数、Accessor、领域 API

基于问题 Q1-Q6 介绍 Pandas 的核心设计思想与用法.

1.4.1 Pandas 的设计哲学

Series、DataFrame 和 Index 从 NumPy 到 Pandas 的设计思想, 其实是在回答一个非常现实的问题:

当数据不再只是“一坨数字”, 而是带着字段名、业务主键、缺失值、时间戳和连接关系的表格时, 我们还能够继续只靠 ndarray 吗?

答案通常是够不够.

NumPy 擅长的是 **同构数值数组** 的高性能计算; 而 Pandas 擅长的是 **带业务语义的异构表格数据** 的组织、筛选、对齐、聚合、连接与时间序列分析.

也就是说:

NumPy 关心的是:

这块连续内存怎样更快地算?

Pandas 关心的是:

这张表里的哪一列代表时间?

哪一列代表金额?

哪些行缺失?

两张表如何按主键对齐?

分组后如何再把结果贴回原表?

这就是 Pandas 的设计出发点:

在 NumPy 的高性能数组之上, 增加标签、索引、缺失值语义、关系代数和时间序列能力, 让“数据分析代码”既能跑得够快, 又能写得像业务逻辑.

Pandas 的三块根基: Series、DataFrame、Index

Series 的本质 Series 可以理解为:

一列带标签的一维数组 = values + index

它不是简单的一维 list, 也不是裸的 ndarray, 而是“数值/对象数组”与“行标签”绑定后的结果.

```
s = pd.Series([10.0, 20.0, 30.0], index=["A", "B", "C"], name="fare")
```

```
print("Series 内容:\n", s)
print("values:", s.values)
print("index :", s.index)
print("name  :", s.name)
print("dtype :", s.dtype)
```

这意味着 Series 的运算不是单纯按位置发生, 而是可以按标签自动对齐:

```
s1 = pd.Series([10, 20, 30], index=["A", "B", "C"])
s2 = pd.Series([1, 2, 3], index=["B", "C", "D"])
```

```
print("按索引对齐相加的结果:\n", s1 + s2)
```

输出会是:

```
A      NaN
B      21.0
C      32.0
D      NaN
dtype: float64
```

因为 Pandas 默认认为:

业务上“B”只能和“B”对齐,“C”只能和“C”对齐,没匹配到的就应当显式表现为缺失值 NaN.

这就是 Pandas 跟 NumPy 在抽象层次上最本质的区别之一.

DataFrame 的本质 DataFrame 可以理解为:

多个共享同一套行索引的 Series 组成的二维表结构

它既可以看作“带列名和行索引的二维表”,也可以看作“按列组织的一组一维数组”.

```
df_demo = pd.DataFrame(
    {
        "fare_amount": [8.5, 12.0, 6.0],
        "passenger_count": [1, 2, 1],
        "pickup_hour": [8, 9, 8],
    },
    index=["trip_1", "trip_2", "trip_3"],
)
```

```
print("DataFrame 内容:\n", df_demo)
print("index      :", df_demo.index)
print("columns   :", df_demo.columns)
print("各列 dtype:")
print(df_demo.dtypes)
```

从物理角度看, DataFrame 更接近 **列式存储**而不是“一个个行对象”.

这件事非常关键,因为数据分析的大量操作都是:

- 对某一列做过滤;
- 对某几列做数值计算;
- 对某一列做分组统计;
- 对时间列做重采样;
- 对主键列做连接.

列式组织意味着 Pandas 可以更高效地把这些操作下压到 NumPy 或扩展数组层面执行.

Index | 的本质 很多初学者把 Index 理解成“行号”,这是不够的.

Index 的真正角色是:

Pandas 中“轴标签”的统一抽象,用于定位、对齐、分组、切片、连接和时间序列操作.

它既可以是默认的 RangeIndex,也可以是:

- 字符串主键组成的 Index
- 时间戳组成的 DatetimeIndex
- 多层标签组成的 MultiIndex

例如:

```
df_idx = pd.DataFrame(
    {
        "fare_amount": [10, 12, 8],
        "passenger_count": [1, 2, 1],
    },
    index=["order_001", "order_002", "order_003"],
)
```

```
print(" 按标签定位 order_002 这一行:\n", df_idx.loc["order_002"])
```

如果把时间列设置为索引:

```
df_time = pd.DataFrame(
    {
        "pickup_count": [120, 150, 180],
    },
    index=pd.to_datetime([
        "2024-01-01 08:00:00",
        "2024-01-01 09:00:00",
        "2024-01-01 10:00:00",
    ]),
)
```

```
print(" 按时间区间切片 (08:00 至 09:00):\n", df_time.loc["2024-01-01 08:00:00":"2024-01-01 09:00:00"])
```

此时 Pandas 就能直接支持基于时间的切片、重采样与滑窗。

Pandas 的设计哲学总结 把上面三者放在一起看, Pandas 的核心设计哲学可以概括为四句话:

1. 在数组之上增加标签语义

NumPy 的世界里, 重点是位置; Pandas 的世界里, 重点是字段名和标签。

2. 在高性能基础上容纳异构数据

现实业务表不是纯 float64 矩阵, 而是金额、字符串、时间、布尔、缺失值混合在一起。

3. 把关系代数和时间序列作为一等公民

groupby、merge、resample、rolling 在 Pandas 里不是边角料, 而是最核心的分析算子。

4. 让分析代码表达业务意图而不是遍历细节

不是反复写:

```
for row in rows:
    ...
```

而是写成:

```
(
    df
    .query("total_amount > 0")
    .assign(pickup_hour=lambda x: x["tpep_pickup_datetime"].dt.hour)
    .groupby("pickup_hour")
    .agg(order_count=("total_amount", "size"))
)
```

这类代码表达的是“业务分析动作”，而不是“解释器怎样逐行跑”。

1.4.2 Q1：这 200 万条数据可靠吗？

这个问题看起来很朴素，但实际上是所有分析的起点。

如果数据本身就不可信：

- 类型错了；
- 时间还没转成真正的时间戳；
- 缺失值没有识别；
- 坐标出现 0 或离谱异常值；
- 金额出现负数或极端脏值；

那么后面所有均值、分组、建模、可视化都会建立在错误基础上。

所以 Q1 的真正目标不是“会几个 API”，而是建立一个习惯：

分析前先做数据可信度诊断 (data validation / data quality check)。

这一单元要解决的 API

技术	在这里解决什么问题
dtype	这一列到底是数值、字符串还是时间？
缺失值	哪些列有 NaN / None / 空洞？
mask	如何批量构造“可信/不可信”的布尔条件？
loc	如何按布尔条件精确筛出异常行？
query	如何用接近 SQL 的写法进行业务过滤？

本质 Q1 本质上做的是三件事：

1. 识别列的数据类型是否符合业务预期；
2. 识别缺失值与异常值；
3. 根据布尔规则把“可信数据子集”筛选出来。

Pandas 在这里的价值是：

把逐行检查，变成整列向量化检查；把“人工目测”，变成“可重复执行的规则”。

底层原理 这一类操作底层主要依赖三件事：

第一，每一列都有明确的 dtype 与缺失值表示语义

Pandas 会根据列类型选择不同的存储与计算路径. 数值列可以直接批量比较; 对象列通常更贵; 时间列转换后才能启用 `.dt`、时间切片和重采样能力.

第二, 布尔掩码本质上是一列 True/False

例如:

```
mask = df["fare_amount"] > 0
```

这个 `mask` 的本质是一个与 `DataFrame` 行数等长的布尔 `Series`, 它像 NumPy 里的布尔索引一样, 但带有索引语义.

第三, `loc` 与 `query` 都是在做“基于条件的行过滤”

- `loc` 更显式, 更适合组合多个条件和精确选列;
- `query` 更接近声明式 DSL, 可读性更强, 在复杂业务表达里很方便.

数据分析中的作用与用法 在真实项目里, Q1 一般会固定形成一个“数据准入规则”阶段:

- 先看 `df.dtypes`
- 再看 `df.isna().sum()`
- 再看描述统计 `describe()` 或分位数
- 再写出异常规则 `mask`
- 最后筛出 `clean_df`

这一步非常像后端系统里的参数校验, 只不过对象从“一次请求”变成了“200 万条记录”.

如果把这道题真正落到 Pandas API 上, 可以按下面顺序思考:

1. 先用 `pd.read_parquet`、`shape`、`dtypes` 认清数据结构和字段类型;
2. 再用 `isna`、数值比较、时间差计算等方式识别缺失值和异常值;
3. 用多个布尔 `mask` 组合成“可信数据”的准入规则;
4. 用 `loc` 抽查异常样本, 用 `query` 产出清洗后的可信子集;
5. 最后用 `np.select` 把异常进一步分类成可统计、可追踪的标签.

```
[19]: # =====
# 0. 读取数据并先看最基础的结构
# =====
TAXI_PARQUET_PATH = "../data/yellow_tripdata_2025-12.parquet"
taxi = pd.read_parquet(TAXI_PARQUET_PATH)

print("数据形状 (行数, 列数):", taxi.shape)
print("各列数据类型:\n", taxi.dtypes)
print("前 5 行数据预览:\n", taxi.head())

# =====
# 1. dtype: 先确认每一列是不是我们以为的类型
# =====
# Parquet 往往会保留 datetime dtype, 但分析前仍建议显式确认
print("转换前各列数据类型:\n", taxi.dtypes)
```

```

# 统一确认上车 / 下车时间列，并顺手构造行程时长（分钟）
taxi["tpep_pickup_datetime"] = pd.to_datetime(
    taxi["tpep_pickup_datetime"],
    errors="coerce",
)
taxi["tpep_dropoff_datetime"] = pd.to_datetime(
    taxi["tpep_dropoff_datetime"],
    errors="coerce",
)
taxi = taxi.assign(
    trip_minutes=(
        taxi["tpep_dropoff_datetime"] - taxi["tpep_pickup_datetime"]
    ).dt.total_seconds() / 60
)

print("转换后各列数据类型:\n", taxi.dtypes)

# =====
# 2. 缺失值诊断
# =====
# isna() 会返回同形状的布尔表；sum() 会把 True 当作 1 来累计
missing_summary = pd.DataFrame(
    {
        "missing_count": taxi.isna().sum(),
        "missing_rate": taxi.isna().mean().mul(100),
    }
).sort_values("missing_count", ascending=False)

print("各列缺失值统计:\n", missing_summary)

# =====
# 3. 使用 mask 批量定义“可信数据”规则
# =====
# 新版黄出租车数据可以直接利用完整字段定义业务边界：
# - 车费与总金额应当 > 0
# - 行程距离应当 > 0
# - 下车时间应晚于上车时间
# - 上下车区域 ID 应当为正整数
mask_positive_fare = taxi["fare_amount"] > 0
mask_positive_total = taxi["total_amount"] > 0
mask_positive_distance = taxi["trip_distance"] > 0
mask_valid_duration = taxi["trip_minutes"] > 0
mask_valid_location = (
    (taxi["PULocationID"] > 0)
    & (taxi["DOLocationID"] > 0)
)

```

```

)

mask_valid = (
    mask_positive_fare
    & mask_positive_total
    & mask_positive_distance
    & mask_valid_duration
    & mask_valid_location
)

print("可信数据占比:", mask_valid.mean())

# =====
# 4. loc: 精确抽取异常行进行排查
# =====
bad_rows = taxi.loc[
    ~mask_valid,
    [
        "fare_amount",
        "total_amount",
        "trip_distance",
        "tpep_pickup_datetime",
        "tpep_dropoff_datetime",
        "trip_minutes",
        "passenger_count",
        "PULocationID",
        "DOLocationID",
    ],
]

print("异常行前 10 条:\n", bad_rows.head(10))

# =====
# 5. query: 用更接近 SQL 的方式做业务筛选
# =====
clean_taxi = taxi.query(
    "fare_amount > 0 and total_amount > 0 and trip_distance > 0 "
    "and trip_minutes > 0 and PULocationID > 0 and DOLocationID > 0"
).copy()

print("query 筛选后数据形状 (行数, 列数):", clean_taxi.shape)

# =====
# 6. 进一步把异常类型分类, 而不是只做二元过滤

```

```
# =====
taxi["quality_flag"] = np.select(
    [
        taxi["fare_amount"] <= 0,
        taxi["total_amount"] <= 0,
        taxi["trip_distance"] <= 0,
        taxi["tpep_pickup_datetime"].isna() | taxi["tpep_dropoff_datetime"].
        isna(),
        taxi["trip_minutes"] <= 0,
        taxi["passenger_count"].isna(),
        taxi["passenger_count"] <= 0,
        (taxi["PULocationID"] <= 0) | (taxi["DOLocationID"] <= 0),
    ],
    [
        "bad_fare",
        "bad_total_amount",
        "bad_trip_distance",
        "bad_datetime",
        "bad_trip_duration",
        "missing_passenger_count",
        "bad_passenger_count",
        "invalid_location_id",
    ],
    default="ok",
)

print("数据质量标记分布:\n", taxi["quality_flag"].value_counts())
```

数据形状 (行数, 列数): (4305006, 20)

各列数据类型:

VendorID	int32
tpep_pickup_datetime	datetime64[us]
tpep_dropoff_datetime	datetime64[us]
passenger_count	float64
trip_distance	float64
RatecodeID	float64
store_and_fwd_flag	object
PULocationID	int32
DOLocationID	int32
payment_type	int64
fare_amount	float64
extra	float64
mta_tax	float64
tip_amount	float64
tolls_amount	float64
improvement_surcharge	float64
total_amount	float64
congestion_surcharge	float64

```

Airport_fee                float64
cbd_congestion_fee         float64
dtype: object

```

前 5 行数据预览:

```

      VendorID tpep_pickup_datetime tpep_dropoff_datetime passenger_count \
0           1  2025-12-01 00:37:08  2025-12-01 00:51:10             1.000
1           2  2025-12-01 00:03:54  2025-12-01 00:19:18             1.000
2           2  2025-12-01 00:40:50  2025-12-01 01:06:54             1.000
3           1  2025-12-01 00:21:30  2025-12-01 00:49:35             2.000
4           2  2025-12-01 00:00:24  2025-12-01 00:03:34             1.000

```

```

      trip_distance RatecodeID store_and_fwd_flag PULocationID DOLocationID \
0           2.400         1.000                N           140           48
1           8.370         1.000                N           138          262
2          15.260         1.000                N           132          255
3          18.400         2.000                N           132           79
4           0.520         1.000                N           239          238

```

```

      payment_type fare_amount extra mta_tax tip_amount tolls_amount \
0           1      14.200  4.250  0.500      4.000      0.000
1           1      33.100  6.000  0.500      7.770      6.940
2           1      57.600  1.000  0.500     12.020      0.000
3           1      70.000  5.000  0.500     10.000      0.000
4           1       5.800  1.000  0.500      1.620      0.000

```

```

      improvement_surcharge total_amount congestion_surcharge Airport_fee \
0           1.000         23.950             2.500      0.000
1           1.000         59.560             2.500      1.750
2           1.000         73.870             0.000      1.750
3           1.000         86.500             2.500      1.750
4           1.000         12.420             2.500      0.000

```

```

      cbd_congestion_fee
0           0.750
1           0.000
2           0.000
3           0.750
4           0.000

```

转换前各列数据类型:

```

VendorID                int32
tpep_pickup_datetime    datetime64[us]
tpep_dropoff_datetime   datetime64[us]
passenger_count         float64
trip_distance           float64
RatecodeID              float64
store_and_fwd_flag      object
PULocationID            int32
DOLocationID            int32

```

payment_type	int64
fare_amount	float64
extra	float64
mta_tax	float64
tip_amount	float64
tolls_amount	float64
improvement_surcharge	float64
total_amount	float64
congestion_surcharge	float64
Airport_fee	float64
cbd_congestion_fee	float64

dtype: object

转换后各列数据类型:

VendorID	int32
tpcp_pickup_datetime	datetime64[us]
tpcp_dropoff_datetime	datetime64[us]
passenger_count	float64
trip_distance	float64
RatecodeID	float64
store_and_fwd_flag	object
PULocationID	int32
DOLocationID	int32
payment_type	int64
fare_amount	float64
extra	float64
mta_tax	float64
tip_amount	float64
tolls_amount	float64
improvement_surcharge	float64
total_amount	float64
congestion_surcharge	float64
Airport_fee	float64
cbd_congestion_fee	float64
trip_minutes	float64

dtype: object

各列缺失值统计:

	missing_count	missing_rate
passenger_count	1195482	27.770
Airport_fee	1195482	27.770
RatecodeID	1195482	27.770
store_and_fwd_flag	1195482	27.770
congestion_surcharge	1195482	27.770
VendorID	0	0.000
mta_tax	0	0.000
cbd_congestion_fee	0	0.000
total_amount	0	0.000
improvement_surcharge	0	0.000
tolls_amount	0	0.000

tip_amount	0	0.000
fare_amount	0	0.000
extra	0	0.000
tpep_pickup_datetime	0	0.000
payment_type	0	0.000
DOLocationID	0	0.000
PULocationID	0	0.000
trip_distance	0	0.000
tpep_dropoff_datetime	0	0.000
trip_minutes	0	0.000

可信数据占比: 0.9406981082024044

异常行前 10 条:

	fare_amount	total_amount	trip_distance	tpep_pickup_datetime	\
5	16.300	30.650	0.000	2025-12-01 00:11:36	
46	-3.000	-7.250	0.040	2025-12-01 00:07:23	
95	3.000	8.500	0.000	2025-12-01 00:33:11	
105	6.500	13.800	0.900	2025-12-01 00:08:50	
106	19.800	25.550	2.540	2025-12-01 00:35:17	
127	-48.500	-61.000	12.430	2025-12-01 00:58:07	
161	-5.100	-7.600	0.420	2025-12-01 00:15:40	
241	34.500	64.730	8.740	2025-12-01 00:57:33	
243	3.000	13.750	0.000	2025-12-01 00:03:47	
265	-7.900	-13.650	1.090	2025-12-01 00:19:31	

	tpep_dropoff_datetime	trip_minutes	passenger_count	PULocationID	\
5	2025-12-01 00:33:17	21.683	1.000	138	
46	2025-12-01 00:07:58	0.583	1.000	132	
95	2025-12-01 00:33:28	0.283	1.000	74	
105	2025-12-01 00:08:50	0.000	1.000	239	
106	2025-12-01 00:35:17	0.000	4.000	143	
127	2025-12-01 01:22:32	24.417	1.000	138	
161	2025-12-01 00:18:34	2.900	1.000	152	
241	2025-12-01 00:57:33	0.000	1.000	138	
243	2025-12-01 00:03:52	0.083	1.000	48	
265	2025-12-01 00:25:18	5.783	1.000	107	

	DOLocationID
5	82
46	132
95	74
105	143
106	170
127	87
161	166
241	107
243	50
265	79

query 筛选后数据形状 (行数, 列数): (4049711, 21)

数据质量标记分布：

```
quality_flag
ok                2946353
missing_passenger_count  1085229
bad_trip_distance    148520
bad_trip_duration    55962
bad_fare             50716
bad_passenger_count   18129
bad_total_amount      97
Name: count, dtype: int64
```

真正需要记住什么？不是“我会 query 和 loc”。而是：

Pandas 的第一价值，是让数据质量检查变成整列、批量、可复现的规则系统，而不是人工抽样拍脑袋。

1.4.3 Pandas: GroupBy、Transform、Merge 对应着问题 Q2 ~ Q4

Q2: 一天中什么时候最忙？Q3: 什么时间段的流水和单位运营时间收入最高？Q4: 哪些上下车区域最热门？

Q2 到 Q4 其实对应了三类极其核心的分析动作：

- Q2: 聚合统计
- Q3: 分组后再回填到原表
- Q4: 把不同来源的数据拼接起来

如果说 Q1 是“先把数据洗干净”，那么 Q2 ~ Q4 就是“开始真正做业务分析”。

[]: